

Lecture Notes: Introduction to Operating Systems

Prof. Mehdi Pirahandeh

September 26, 2025

Contents

1	Introduction	3
2	Computer System Structure	4
2.1	What Operating Systems Do	5
2.2	Key OS Functions	6
2.3	Operating System Services	8
2.3.1	User-Oriented Services	8
2.3.2	System-Oriented Services	9
2.4	User and OS Interface	10
2.5	System Calls	10
2.5.1	Examples of System Calls	10
2.5.2	Formula: Average System Call Time	10
2.6	System Services	11
2.7	Linkers and Loaders	11
2.7.1	Linker	11
2.7.2	Loader	11
2.8	Why Applications are OS-Specific	11
2.9	Design and Implementation of OS	11
2.9.1	Goals	11
2.9.2	Policy vs Mechanism	12
2.10	Operating System Structures	12
2.10.1	Monolithic	12
2.10.2	Layered	12

2.10.3	Microkernel	12
2.10.4	Modules	12
2.10.5	Hybrid	12
2.11	Examples of OS Architectures	12
2.12	Building and Booting an OS	13
2.12.1	Formula: Boot Time	13
2.13	Debugging and Performance Tuning	13
2.14	Debugging	13
2.14.1	Performance Tuning	13
2.15	Tracing Tools	13
3	Process Concept	14
3.1	Definition	14
3.2	Formulas	14
3.3	Process State	15
3.4	Process Control Block (PCB)	15
3.5	Threads	16
3.6	Process Scheduling	16
3.7	Goals	16
3.7.1	Scheduling Queues	16
3.7.2	Formula: CPU Utilization	16
3.7.3	Context Switch	16
3.8	Operations on Processes	16
3.8.1	Process Creation	16
3.8.2	Process Termination	17
3.9	Interprocess Communication (IPC)	17
3.10	Motivation	17
3.10.1	Models of IPC	17
3.10.2	Producer-Consumer Problem	17
3.11	Message Passing	18
3.11.1	Direct vs Indirect	18
3.11.2	Synchronization	18
3.11.3	Buffering	18
3.12	Examples of IPC	18
3.12.1	POSIX Shared Memory	18
3.12.2	Mach IPC	18
3.12.3	Windows IPC	19
3.13	Pipes	19

3.14	Client-Server Communication	19
3.14.1	Sockets	19
3.14.2	Remote Procedure Calls (RPC)	19
4	Formulas and Descriptions	19
4.1	Storage Units	19
4.2	CPU Utilization Formula	19
4.3	Throughput Formula	20
4.4	Turnaround Time Formula	20
4.5	Response Time Formula	20
5	Summary Table of Important Concepts	22
6	Conclusion	23

1 Introduction

An **Operating System (OS)** is software that acts as an intermediary between computer hardware and users. It ensures that resources like CPU, memory, and input/output devices are used efficiently and fairly.

Computer System Structure A computer system can be divided into four main components:

- **Hardware** – CPU, memory, I/O devices.
- **Operating System** – Controls and coordinates use of hardware.
- **Application Programs** – Examples: word processors, games, browsers.
- **Users** – People, machines, or other computers.

What Operating Systems Do Depending on perspective:

- **For Users:** Provide convenience, usability, and performance.
- **For System Management:** Allocate resources, ensure fair sharing, handle errors.

Key OS Functions

1. **Process Management:** Creating, scheduling, synchronizing processes.

2. **Memory Management:** Deciding which program/data should reside in RAM.
3. **File System Management:** Organizing, reading, and writing files.
4. **I/O Management:** Managing devices and input/output requests.
5. **Protection & Security:** Preventing unauthorized access.

2 Computer System Structure

A computer system is like a team, where each member has a specific role to make the entire machine work smoothly. It can be divided into four main components:

Hardware

The hardware forms the physical backbone of the system. It includes:

- **CPU (Central Processing Unit):** Often called the “brain” of the computer, it executes instructions and performs calculations.
- **Memory:** Stores both instructions and data temporarily while tasks are running.
- **Input/Output (I/O) Devices:** Components like keyboards, mice, screens, printers, and network cards that allow interaction with the system.

Operating System (OS)

The OS is like a manager. It controls and coordinates the hardware resources. Without the OS, the hardware is difficult to use directly. For example, instead of writing a program that talks directly to a keyboard, programmers can rely on the OS to handle the keyboard input.

Application Programs

These are the tools and software that users work with to perform specific tasks. Examples include:

- Word processors such as Microsoft Word or Google Docs.
- Games and entertainment software.
- Web browsers like Chrome or Firefox.
- Database systems that store and manage data.

Users

Users can be people, machines, or even other computers connected through a network. A user's requests are passed to the OS, which then communicates with hardware to fulfill them.

Thus, the structure can be imagined as layers: at the bottom is **hardware**, above it the **operating system**, then **applications**, and at the top the **users**. This layered approach makes systems flexible and easier to use.

2.1 What Operating Systems Do

The operating system performs many tasks, but its role depends on the perspective:

For Users

Users typically want:

- Convenience – The computer should be easy to use, with simple commands and graphical interfaces.
- Performance – Programs should run quickly and smoothly.
- Reliability – The system should not crash frequently.

For example, when you open a web browser, you expect it to launch quickly without worrying about how memory or CPU are allocated.

For System Management

From the perspective of the system, the OS must:

- Allocate resources such as CPU time, memory space, and input/output channels.
- Ensure fair sharing between multiple applications and users.
- Handle errors and recover from failures.

Think of the OS as a referee in a game, ensuring all players (programs) get a fair chance to use the resources without conflicts.

*Different Types of Systems

- **Personal computers:** Focus on user-friendliness and performance.
- **Servers:** Must manage many users and ensure stability.
- **Mobile devices:** Optimize battery life and usability.
- **Embedded systems:** Often run with no user interface, like in washing machines or cars.

2.2 Key OS Functions

An operating system has several core responsibilities that ensure smooth operation of the computer.

1. Process Management

A **process** is a program in execution. Process management includes:

- Creating and deleting processes.
- Scheduling which process runs on the CPU and when.
- Synchronizing processes so that they work together without errors.

Formula: CPU Utilization

$$\text{CPU Utilization} = \frac{\text{Time CPU is Busy}}{\text{Total Time}} \times 100\%$$

2. Memory Management

Programs and data must reside in memory to be executed. The OS decides:

- Which part of memory is in use and by which process.
- Which data should be moved in or out of memory.
- How to allocate and free up memory space.

This ensures efficient use of RAM and avoids crashes due to memory shortages.

3. File System Management

Information on a computer is organized into files. The OS:

- Creates, deletes, and organizes files into directories.
- Maps files onto physical storage like hard disks or SSDs.
- Provides security and backup to protect files.

4. I/O Management

Every input/output device has different rules for operation. The OS hides these details and provides a uniform interface. Tasks include:

- Managing data transfer between memory and I/O devices.
- Handling device drivers.
- Buffering and caching to improve speed.

5. Protection and Security

Protection ensures that processes cannot interfere with each other's memory or resources. Security defends against external threats like viruses or hackers. Techniques include:

- User IDs and passwords for authentication.
- Permissions and access control.

- Preventing privilege escalation (unauthorized gaining of higher rights).

In short, the OS is like a multi-talented administrator: it manages processes, controls memory, organizes files, handles devices, and protects the system. Without the OS, users would have to manually control every piece of hardware, making modern computing practically impossible.

2.3 Operating System Services

Operating Systems (OS) provide the foundation for all modern computing. They are responsible for managing hardware, running applications, and ensuring user interaction is smooth. This lecture focuses on OS services, system calls, structures, loaders, debugging, and performance tuning. The goal is to explain not only technical concepts but also why they matter, with examples relevant to high school students. Operating systems provide an **execution environment** for programs and a set of services to users. These services can be categorized into two groups: those helpful to users, and those ensuring efficient system operation.

2.3.1 User-Oriented Services

- **User Interface (UI)** – Can be:
 - Command-Line Interface (CLI) – text-based (e.g., MS-DOS, Linux shell).
 - Graphical User Interface (GUI) – windows, icons, mouse-based interaction.
 - Touch-based interface – used in smartphones and tablets.
- **Program Execution** – Loading programs into memory and running them.
- **I/O Operations** – Reading and writing from files or devices.
- **File-System Manipulation** – Creating, deleting, searching, and securing files.
- **Communications** – Message passing between processes or computers.
- **Error Detection** – Identifying and correcting errors in CPU, memory, I/O, or user programs.

2.3.2 System-Oriented Services

- **Resource Allocation** – Fair distribution of CPU cycles, memory, files, and devices.
- **Logging** – Tracking system usage.
- **Protection and Security** – Preventing unauthorized access, ensuring safe execution of processes.

Service	Description
User Interface	CLI, GUI, or touch-based for interacting with the OS.
Program Execution	Load, run, and terminate applications.
I/O Operations	File or device input/output operations.
File System	Organizing files and directories, access control.
Communications	Shared memory or message passing between processes.
Error Detection	Detect and handle errors to maintain consistency.
Resource Allocation	CPU, memory, storage management among processes.
Protection & Security	Control user access, defend against attacks.

2.4 User and OS Interface

Interaction between users and the OS happens via:

1. **CLI (Command-Line Interface)** – direct text commands.
2. **GUI (Graphical User Interface)** – visual and intuitive.
3. **Touchscreen and Voice Interfaces** – gestures or speech input.

For example, Windows uses both GUI and a command shell, while Linux allows both terminal (CLI) and graphical environments like GNOME or KDE.

2.5 System Calls

System calls provide the **programming interface** to OS services. They allow user programs to request services from the kernel.

2.5.1 Examples of System Calls

- **Process Control** – create, terminate, wait, signal.
- **File Management** – create, open, read, write, close files.
- **Device Management** – request, release, read, write devices.
- **Information Maintenance** – get time/date, system info.
- **Communication** – message passing, shared memory.
- **Protection** – set permissions, deny access.

2.5.2 Formula: Average System Call Time

If T_{user} is time in user mode and T_{kernel} is time in kernel mode:

$$T_{syscall} = T_{user} + T_{kernel}$$

2.6 System Services

System services are programs that make OS functionality accessible:

- File manipulation tools (copy, move, search).
- Status tools (date, time, memory usage).
- Editors and debuggers for development.
- Communication tools (chat, email, file transfer).
- Background services like printing, scheduling, logging.

2.7 Linkers and Loaders

2.7.1 Linker

Combines object files into a single executable, resolves references, and links libraries.

2.7.2 Loader

Places the program into memory, relocates addresses, and prepares it for execution.

2.8 Why Applications are OS-Specific

Applications compiled for one OS often cannot run on another because:

- System calls differ across OSes.
- File formats and binary standards vary.

Solutions: use portable languages (Java, Python) or compile separately for each OS.

2.9 Design and Implementation of OS

2.9.1 Goals

- **User goals** – Easy, fast, reliable, safe.
- **System goals** – Flexible, efficient, maintainable.

2.9.2 Policy vs Mechanism

- **Policy:** What to do. Example: Interrupt after 100 ms.
- **Mechanism:** How to do it. Example: Hardware timer.

2.10 Operating System Structures

2.10.1 Monolithic

All OS functions in one large program (UNIX).

2.10.2 Layered

Organized into layers, each layer uses lower ones.

2.10.3 Microkernel

Moves non-essential functions into user space, communicates via message passing.

2.10.4 Modules

OS components as loadable modules.

2.10.5 Hybrid

Combines multiple approaches (Windows, macOS, Linux).

2.11 Examples of OS Architectures

- **Linux:** Monolithic + modular.
- **Windows:** Hybrid with microkernel features.
- **macOS:** Hybrid with Mach microkernel + BSD Unix.
- **iOS/Android:** Based on UNIX/Linux, layered with APIs and run-times.

2.12 Building and Booting an OS

1. Write source code.
2. Configure OS.
3. Compile kernel.
4. Install kernel and modules.
5. Boot system via boot loader (e.g., GRUB, UEFI).

2.12.1 Formula: Boot Time

$$T_{boot} = T_{BIOS/UEFI} + T_{loader} + T_{kernel}$$

2.13 Debugging and Performance Tuning

2.14 Debugging

- Application failures produce **core dumps**.
- OS failures produce **crash dumps**.

2.14.1 Performance Tuning

$$\text{CPU Utilization} = \frac{\text{Busy Time}}{\text{Total Time}} \times 100\%$$

Tools: `top`, Task Manager, profilers.

2.15 Tracing Tools

- `strace` – traces system calls.
- `gdb` – debugger.
- `perf` – Linux performance tools.
- `tcpdump` – network packet tracing.
- BCC/eBPF – advanced Linux tracing framework.

Week 3 expanded the students' understanding of how operating systems provide services, interact with users, manage execution, and ensure performance. By exploring system calls, loaders, OS design, booting, and debugging, we see the OS as not only a technical foundation but also as a crucial enabler of modern computing

3 Process Concept

This lecture explores the **concept of processes** in operating systems. Processes are the fundamental units of execution, and understanding them is crucial for learning how computers actually run programs. We also study how processes are scheduled, created, terminated, and how they communicate with each other through mechanisms such as **shared memory**, **message passing**, **pipes**, and **sockets**.

3.1 Definition

A **process** is a program in execution. While a program is a passive entity stored on disk (like a Word file or executable), a process is an active entity that includes:

- The **text section** (program code).
- The **program counter** and **CPU registers**.
- A **stack** containing function parameters, return addresses, and local variables.
- A **data section** with global variables.
- A **heap** with dynamically allocated memory during runtime.

3.2 Formulas

Turnaround Time:

$$T_{turnaround} = T_{completion} - T_{arrival}$$

Response Time:

$$T_{response} = T_{first_response} - T_{request}$$

3.3 Process State

A process changes state as it runs:

- **New** – Being created.
- **Running** – Instructions are executed.
- **Waiting** – Process waits for an event (I/O).
- **Ready** – Waiting for CPU allocation.
- **Terminated** – Execution finished.

State	Description
New	Process is being initialized.
Running	CPU is actively executing instructions.
Waiting	Process paused until event occurs (e.g., I/O complete).
Ready	Waiting to be assigned a CPU core.
Terminated	Process has completed execution.

3.4 Process Control Block (PCB)

Every process is represented by a **PCB**, which contains:

- Process state.
- Program counter.
- CPU registers.
- CPU scheduling information (priority, queue pointers).
- Memory management details.
- Accounting information (time used, limits).
- I/O status (devices, open files).

3.5 Threads

So far, each process has been assumed to have a single **thread of execution**. Threads allow multiple sequences of execution within a single process. For example, in a web browser:

- One thread handles rendering.
- Another handles network requests.
- Another manages user input.

3.6 Process Scheduling

3.7 Goals

The OS must schedule processes to maximize CPU utilization and throughput.

3.7.1 Scheduling Queues

- **Ready Queue:** All processes in main memory waiting for CPU.
- **Wait Queue:** Processes waiting for I/O.

3.7.2 Formula: CPU Utilization

$$U_{CPU} = \frac{\text{CPU Busy Time}}{\text{Total Time}} \times 100\%$$

3.7.3 Context Switch

Switching CPU control from one process to another requires saving the old process state and loading the new process state. **Context Switch Time** is pure overhead: the CPU performs no useful work during it.

3.8 Operations on Processes

3.8.1 Process Creation

A parent creates child processes, forming a **process tree**. Examples:

- `fork()` in UNIX creates a duplicate process.
- `exec()` replaces the process with a new program.

3.8.2 Process Termination

- A process ends with `exit()`.
- Parent may use `wait()` to collect child's status.
- Cascading termination occurs when parent and children are all terminated.

3.9 Interprocess Communication (IPC)

3.10 Motivation

Processes may be independent or cooperating. Cooperation allows:

- Information sharing.
- Speedup of computation.
- Modularity.
- Convenience.

3.10.1 Models of IPC

- **Shared Memory** – Processes directly read/write into common memory space.
- **Message Passing** – Processes exchange data using messages.

3.10.2 Producer-Consumer Problem

Classic example of IPC:

- Producer generates items and places them in a buffer.
- Consumer retrieves items.

Formula: Bounded Buffer Size

$$N = B - 1$$

where B is total buffer slots, N is usable slots.

3.11 Message Passing

3.11.1 Direct vs Indirect

- Direct: send/receive with process names.
- Indirect: send/receive via mailboxes (ports).

3.11.2 Synchronization

- Blocking send/receive – synchronous communication.
- Non-blocking send/receive – asynchronous.

3.11.3 Buffering

- Zero capacity – sender waits until receiver ready.
- Bounded capacity – queue size n .
- Unbounded capacity – infinite message queue.

3.12 Examples of IPC

3.12.1 POSIX Shared Memory

Processes use:

```
1 shm_fd = shm_open(name, O_CREAT|O_RDWR, 0666);  
2 ftruncate(shm_fd, 4096);  
3 ptr = mmap(0, SIZE, PROT_READ|PROT_WRITE, MAP_SHARED, shm_fd,  
    0);
```

3.12.2 Mach IPC

Message-passing based, even system calls are messages.

3.12.3 Windows IPC

Advanced Local Procedure Calls (LPC) use ports for client-server communication.

3.13 Pipes

- **Ordinary Pipes:** Parent-child communication, unidirectional.
- **Named Pipes:** More powerful, bidirectional, no parent-child needed.

3.14 Client-Server Communication

3.14.1 Sockets

Sockets provide communication endpoints, defined by IP + Port.

3.14.2 Remote Procedure Calls (RPC)

A client invokes a procedure on a remote server as if it were local.

Processes are the lifeblood of modern operating systems. By learning how they are represented, created, scheduled, and communicate, students gain a deep understanding of how computing systems multitask and support multiple users and applications at once.

4 Formulas and Descriptions

4.1 Storage Units

The basic unit of storage is a **bit** (0 or 1). Higher units:

$$1 \text{ Byte} = 8 \text{ bits}$$

$$1 \text{ KB} = 1024 \text{ Bytes}, \quad 1 \text{ MB} = 1024^2 \text{ Bytes}$$

$$1 \text{ GB} = 1024^3 \text{ Bytes}, \quad 1 \text{ TB} = 1024^4 \text{ Bytes}$$

4.2 CPU Utilization Formula

The OS aims to maximize CPU usage:

$$\text{CPU Utilization} = \frac{\text{Time CPU is Busy}}{\text{Total Time}} \times 100\%$$

4.3 Throughput Formula

Number of processes completed per unit time:

$$\text{Throughput} = \frac{\text{Number of Processes Completed}}{\text{Total Time}}$$

4.4 Turnaround Time Formula

The total time taken for a process:

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

4.5 Response Time Formula

For interactive systems:

$$T_{\text{response}} = T_{\text{first_response}} - T_{\text{request}}$$

5 Summary Table of Important Concepts

Concept	Description
Operating System	Software layer between hardware and user, ensuring efficiency.
Process	A program in execution, requiring CPU, memory, and I/O.
Memory Management	OS decides which data and programs are in RAM.
File System	Organizes data into files/directories, provides access methods.
CPU Utilization	Percentage of time CPU is working.
Throughput	Number of processes completed per unit time.
Turnaround Time	Time taken from process submission to completion.
Response Time	Time between request submission and first response.
Security	Mechanisms to protect against unauthorized access or malware.
System Calls	Bridge between user programs and kernel.
System Services	Utility programs for file, communication, debugging.
Linkers & Loaders	Prepares programs for execution.
OS Structures	Monolithic, layered, microkernel, hybrid.
Boot Process	BIOS/UEFI → Loader → Kernel.
Debugging	Finding errors, analyzing logs, dumps.
Performance Tuning	Optimize CPU, memory, I/O utilization.
Tracing Tools	Monitor system calls, processes, packets.
Process	Program in execution with stack, heap, and registers.
Process State	New, Ready, Running, Waiting, Terminated.
PCB	Stores process info: registers, memory, I/O, scheduling.
Scheduling	Decides which process runs on CPU.
Context Switch	Saving/loading state when CPU switches.
IPC	Mechanisms for process cooperation.
Shared Memory	Direct communication via memory space.
Message Passing	Communication via send/receive.
Pipes	Conduits for process communication.
Sockets	Network endpoints defined by IP + Port.
RPC	Remote invocation of server-side procedures.

6 Conclusion

Operating systems are fundamental to modern computing. They manage resources, ensure fairness, and provide the foundation for applications. Understanding the basics of OS prepares students to study more advanced computer science topics such as concurrency, scheduling, and distributed systems.